# APPLICATION DEVELOPMENT PRACTICES

Dr.Lê Hùng Tiến

VLU

APPLICATION DEVELOPMENT PRACTICES

# Using Your Configuration Management Processes Part 2

# Director's Overview

- Overview of Java Development Tools Used in this Course
  - eclipse
  - svn
  - Ant
  - CruiseControl (not used in this course)
  - JUnit

# JUnit

- JUnit is an open source automated Java software testing environment that:
  - Is used by programmers to design and implement automated unit tests
  - Helps implement Test-Driven Development (TDD)
- http://www.junit.org

# eclipse – Test Case

Account.java | BankTest.java

```
import junit.framework.TestCase;

public class BankTest extends TestCase {

    public void testAccountOwner() throws Exception {
        Account account = new Account("12345", "Joe Blow");
        assertEquals("Joe Blow", account.getName());
    }

    public void testDeposit() throws Exception {
        Account account = new Account("12345", "Joe Blow");
        account.deposit(150.22);
        assertEquals(150.22, account.getBalance(), 0.0);
    }
}
```

# CruiseControl

- CruiseControl is open source software written in Java that builds (ant) and tests (JUnit) your software product according to the parameters you give it
  - Every hour
  - Specific time every day (before the developers get to work, for example)
  - When any new code is checked in
- Sends email notifications of build status
- Maintains build history and content website
- cruisecontrol.sourceforge.net/

# CruiseControl

- Requires a separate build machine

# Can You Afford a Build Machine?

- Can your team afford a machine dedicated to automatically building and testing your software on a regular interval?
  - Assume that a ten-person development team costs your company $500/ hour
  - If that team spends two less hours debugging integration problems over the life of the project, you've paid for a build machine
- Every day your team is fighting integration and quality problems at the end of your project is another day your product is late to market
  - You can't afford not to have a dedicated build machine
  - Is your competition using automated building & testing?

# CruiseControl

- Automated Builds
  - Push a button
- Scheduled Builds
  - Don’t need to push any buttons

# For Continuous Integration ...

- SCM system
  - To ensure latest code changes are included
- Automated or (better) scheduled builds
  - To detect and resolve build issues early and often
- Automated unit tests
  - To detect and resolve unit and integration issues early
- (Not really part of Continuous Integration, but some teams also like to automate some acceptance tests)

# Developer's Goals

- Deliver working production code
  - Measurable quality
  - Resulting in happy customers
- Be productive
  - Intelligent use of time
  - Address issues associated with software changing over time
- Work smarter, not harder
  - Processes
  - Tools

# Three Types of Code Changes

- Fix defects
- New requirements
- Refactoring
  - Refactoring is NOT the same as fixing defects
  - You end up with exactly the same functionality as before, just (hopefully) simpler code

# Making Code Changes

- If you make even a simple change, how do you know that you haven't broken anything?

# Making Code Changes

- You **MUST** have automated unit tests
- Imagine a large application with many people changing it
  - Adding new features
  - Fixing defects
  - Refactoring code
- Continuous Integration will help you address most of your integration problems early – if you build your automated test cases correctly

# Automated Unit Test Cases

- Create your unit test cases first, then change the code until the test cases pass
  - Adding new features
  - Fixing defects

# SCM & Continuous Integration Perceived Weaknesses

- Too much overhead (time and money)
  - Overhead is much less than time and effort required to fix integration issues late in the project
- Requires discipline by the project team
  - This is a good thing
  - Soft measures of project maturity:
    - Who does your builds?
    - How easy it is for any developer to make a change, test it and commit the change for other developers to use

# Summary – 1

- Build, compile and debug software
  - eclipse
- Build, compile and debug software where source code is stored in a shared repository
  - eclipse
  - svn
- Build, compile and debug software where source code is stored in a shared repository and builds are automated
  - eclipse
  - svn
  - Ant

# Summary – 2

- [Continuous Integration] Build, compile and debug software where source code is stored in a shared repository and builds and unit testing are automated
  - eclipse
  - svn
  - Ant
  - JUnit